



PDF Download
3472538.3472541.pdf
07 March 2026
Total Citations: 3
Total Downloads: 354

 Latest updates: <https://dl.acm.org/doi/10.1145/3472538.3472541>

RESEARCH-ARTICLE

Leveraging Reinforcement Learning and WaveFunctionCollapse for Improved Procedural Level Generation

MATHIAS BABIN, Western University, London, ON, Canada

MICHAEL KATCHABAW, Western University, London, ON, Canada

Open Access Support provided by:

Western University

Published: 21 October 2021

[Citation in BibTeX format](#)

FDG'21: The 16th International
Conference on the Foundations of Digital
Games 2021

August 3 - 6, 2021
QC, Montreal, Canada

Leveraging Reinforcement Learning and WaveFunctionCollapse for Improved Procedural Level Generation

Mathias Babin
University of Western Ontario
London, Ontario, Canada
mbabin2@uwo.ca

Michael Katchabaw
University of Western Ontario
London, Ontario, Canada
mkatchab@uwo.ca

ABSTRACT

This work presents a novel approach to training Reinforcement Learning (RL) agents to serve as heuristics for the WaveFunctionCollapse (WFC) algorithm in the production of procedurally generated video game levels. While the algorithm's original minimal entropy heuristic is sufficient for constructing levels that look visually appealing, it is often the case that these levels suffer when playability is concerned. The approach presented in this work involves replacing this heuristic with a set of deep neural networks trained using RL to direct the algorithm in the construction of playable levels for the original Super Mario Bros. (SMB). We evaluate the performance of our models using a game-playing A*-based agent provided by the Mario AI Competition Framework and designate a simple reward function which reflects the quality of generated levels based on the A* agent's ability to successfully navigate them. Results using this approach show an increase in the percentage of playable levels generated using our learned heuristics over those using minimal entropy.

CCS CONCEPTS

• **Applied computing** → **Computer games**; • **Theory of computation** → **Reinforcement learning**; *Constraint and logic programming*; Random walks and Markov chains.

KEYWORDS

procedural content generation, wavefunctioncollapse, reinforcement learning

ACM Reference Format:

Mathias Babin and Michael Katchabaw. 2021. Leveraging Reinforcement Learning and WaveFunctionCollapse for Improved Procedural Level Generation. In *The 16th International Conference on the Foundations of Digital Games (FDG) 2021 (FDG'21), August 3–6, 2021, Montreal, QC, Canada*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3472538.3472541>

1 INTRODUCTION

Procedural Content Generation (PCG) refers to the generation of any game content through an algorithm that requires limited to no human input [19]. It is often used to generate functional game

content such as levels, rules, and interactable objects, as well as non-essential elements such as music, textures, meshes, and narrative.

In today's commercial game development, PCG is commonly used to lower development costs through a reduction in the number of artists and level designers needed for a project as well as providing an increase to player satisfaction with the game through more substantial replay value; however, it is also crucial to entire genres such as rogue-likes which typically procedurally generate an entirely new environment each time the player dies, or sandbox and survival games which often rely on players exploring massive procedurally generated worlds in order to gather the resources they need to survive. Academic research in the field has contributed solutions to many of these problems, but also explores how PCG can generate content such as puzzles [11], narratives [3], and systems which adapt to the player's behaviour [13].

Considering the breadth of research being conducted in this field, this work will be contributing to the most common form of PCG found in both academic research and modern commercial game development: the generation of novel game environments. In particular, we will be presenting a method for learning heuristics that optimize the playability of game levels generated using the WaveFunctionCollapse (WFC) algorithm. The specific game discussed in this paper is the original *Super Mario Bros.* (SMB) (Nintendo, 1985), however, this method can be applied to any game which can be both represented using WFC, and has a game-playing agent readily available.

Our approach involves training Deep Convolutional Neural Networks (DCNNs) to replace the original minimal entropy heuristic in the WFC algorithm; while minimal entropy is sufficient for constructing levels which look visually appealing, it also has a tendency to create levels that are not playable and would only result in frustrating the player. Our solution to this problem is to involve a game-playing agent provided by the Mario AI Competition Framework [9] as a means of content evaluation in the training of our new heuristics; and once learned, we find that these heuristics successfully outperform the original minimal entropy heuristics in guiding the WFC algorithm towards the construction of playable levels.

2 BACKGROUND AND RELATED WORK

2.1 PCG Methods

Most applications of PCG can be categorized as either search-based [24], solver-based [20], constructive [19], Procedural Content via Machine Learning (PCGML) [23], and most recently, Procedural Content Generation via Reinforcement Learning (PCGRL) [12], with numerous academic contributions made to each of them; many of which feature SMB as their experimental game environment

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FDG'21, August 3–6, 2021, Montreal, QC, Canada

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8422-3/21/08...\$15.00

<https://doi.org/10.1145/3472538.3472541>

of choice. This is perhaps due to the game’s cultural popularity, availability of level data, and game-playing agents through the Mario AI framework. We have seen the application of search-based methods such as [1] and [4] which make use of Genetic Algorithms to evolve levels for SMB. In terms of constructive methods, [18] used a simple context-free grammar to build their SMB levels before optimizing certain structural characteristics using Grammatical Evolution. Finally, as an example of PCGML, [25] generates SMB levels using a Generative Adversarial Network (GAN) trained on the game’s first level.

The complication that arises when classifying this work’s method of PCG involves both the classification of the WFC algorithm itself as well as the machine learning methods used to train our heuristics. [14] suggests that WFC is an example of a search-based algorithm, however, we would argue that it is most similar to a constructive or solver-based method as the algorithm only produces one output per run unlike search-based methods which produce many outputs in a generate-and-test loop. For the purposes of this paper, we will consider it a solver-based method due to the fact that it is an example of a constraint solving algorithm [10] although we acknowledge that many other examples of solver-based PCG rely on traditional constraint satisfaction techniques such as those found in logic programming such as SAT or Answer Set Programming (ASP).

2.2 PCGML and PCGRL

The main distinction between methods like search/solver and methods like PCGML/PCGRL is what is being searched for: search and solver-based methods are those which search through content-space to find their content directly, while PCGML and PCGRL search for policies that ultimately generate the content. To further distinguish between the latter machine learning methods, we must consider how they learn their policies. PCGML typically relies on supervised learning in order to find a sufficient policy, this means that the optimization strategy typically relies on a set of training data in order to learn the structural features of a level, while the reinforcement learning (RL) agents used in PCGRL learn their policies through interactions with their environment and do not require any form of supplementary training data.

With these distinctions in mind, we would characterize the methods presented in this work as those belonging to PCGRL. While we do not use traditional optimization algorithms, such as Q-Learning or Proximal Policy Optimization (PPO), we instead choose the Evolutionary Strategy (ES) presented in [16] as a class of black box optimization algorithms found to be a viable alternative to these popular policy gradient and Markov Decision Problem-based techniques. This said, in the future work of this paper, we will discuss how our approach could be trained by any of these popular optimization algorithms such as Q-Learning, DQN, or PPO as it does still adhere to the standard RL framework.

2.3 Wave Function Collapse

The WFC algorithm was first introduced as a method for texture synthesis and tilemap generation by author Maxim Gumin [6], and since its inception, a number of commercially released games such as *Bad North* [22] and *Caves of Qud* [5] have utilized the algorithm

for the generation of their games’ levels. It is also the case that a number of academic works have experimented with modifications and novel applications of the algorithm such as [17] which modified the algorithm to generate paths for Non-playable characters, or [14] which combined WFC with other constructive methods for use in procedural dungeon generation systems.

The version of the algorithm used in this work is an example-driven image generation algorithm that extracts local patterns from the input image (or tilemap) and generates an output image by growing a partial assignment that eventually renders the total assignment back into an image in the same format as the input [10]. For the purposes of gaining a cursory understanding we will be providing an explanation of a simplified version of the algorithm, which begins by scanning a tilemap for the extraction of patterns of size $N \times N$ which will inform it on which tiles can be placed next to one another and which cannot; these are our constraints. Next, we instantiate a 2D grid, which is referred to as a *wave*, with dimensions equal to our desired output. At each step, the algorithm attempts to select the tile that not only adheres to the constraints of its neighbours, but is also the most likely to appear based on what it has observed in the original sample. The process of choosing a specific tile at a given wave position is called *collapsing*, as the number of possible tiles that could have been placed is collapsed down into only a single option. Once a position in the wave has been collapsed, the algorithm performs a constraint propagation step where the possible tiles of all neighbouring positions of the most recently collapsed position are reduced down to only those tiles which conform with the current state of the wave. This process of collapsing wave positions and propagating constraints continues until all positions have been collapsed or until a conflict occurs. Should the algorithm encounter a position where no object satisfies all neighbouring constraints, i.e. there is a conflict, one has to decide whether to backtrack to a point in the algorithm where there were no conflicts, or simply restart and try the process again. Finally, it should be noted that the process of selecting a wave position to be collapsed and the tile that is ultimately selected is controlled by a heuristic. As originally described, WFC uses a minimal entropy heuristic which finds the wave position with “the tightest or smallest domain after propagation” [10]. This heuristic attempts to minimize the number of conflicts that occur by collapsing wave positions that eliminate the fewest possible legal pattern assignments that can be made in the future.

2.4 Reinforcement Learning and PCGRL

In the standard RL agent-environment interaction loop, agents are provided with the environment’s current state s_t and based on this state perform an action a_t . Next, the environment returns a new state s_{t+1} , and a reward r_{t+1} which is used for evaluating the agent’s performance. The goal of all RL agents is to maximize the cumulative reward they receive at each time step t . The PCGRL framework expands on this by introducing a parameter called *Change Percentage* that monitors how much of the initial level the agent is allowed to modify before halting. For the approach presented in this work, we will not be using this parameter as the number of steps required to generate one of our levels is bounded by the dimensions of the output and number of possible tile patterns that can be placed.

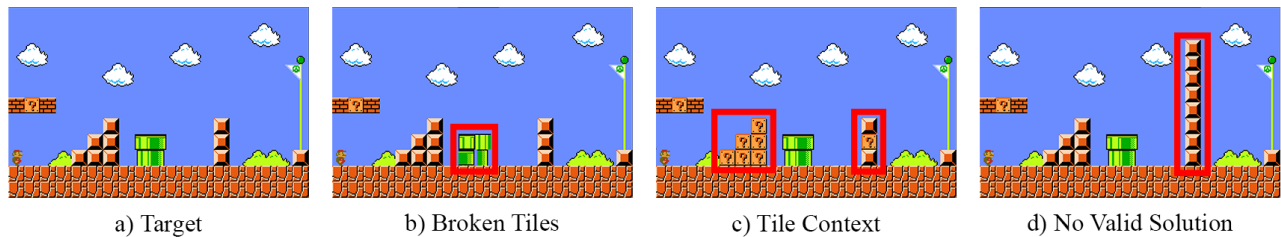


Figure 1: A desirable target output for our system (a) as well as three possible issues encountered when generating tile-based levels: b shows a scenario where tiles meant to form larger composite objects are not placed properly, c shows tiles placed in inappropriate contexts, and d shows how tiles can be placed such that there is no valid solution to the level.

As presented in the original PCGRL paper, the formulation of level generation as a RL problem involves framing level design as an iterative process where instead of taking actions to play a game, an agent’s actions are to modify existing structures in a level. Beyond the agent’s actions, a typical RL scenario would reward an agent for playing a game well, usually based off of the game’s internal scoring mechanism, but in a PCGRL task, we instead reward the agent’s ability to construct levels that meet a certain set of design criteria; in most cases, this evaluation must be conducted via an independent game-playing agent such as those provided in the Mario AI framework, or the General Video Game AI framework [15].

3 APPROACH

As a texture synthesis algorithm, WFC relies on heuristics to guide its generation, most notably a stochastic minimal entropy heuristic. This work aims to replace this heuristic with a set of RL agents who instead guide the algorithm in the generation of functionally playable levels for the original SMB. Where minimal entropy would generate levels that look visually similar to an original sample, our agents are motivated to generate levels that contain valid solution paths, i.e. they are guiding the algorithm to produce levels that are always playable. We consider the key contribution of this work as the integration of WFC into the environment of a RL task, which to our knowledge, has never been attempted.

3.1 WaveFunctionCollapse For Level Generation

When generating levels for tile-based games such as *SMB*, there are a number of issues that should be considered. The left-most example of Figure 1 shows an ideal target level that we would like our own generated levels to resemble, followed by three levels which demonstrate issues which may occur when performing tile-based level construction. The first of these issues can be seen in example *b*, which shows how a level generator may easily encounter issues when assembling larger composite tiles, such as warp pipes, by failing to place a set of smaller tiles in their proper sequence. Next, *c* shows how procedural generation systems may place tiles in inappropriate contexts; in this example, question blocks were erroneously repurposed into a staircase or as part of a wall. It is important to note that this mistake will technically function in the exact same manner as the original target example, however, from a

design perspective, it is violating the expectations players will have for how to interact with these tiles, and therefore is something we should try to avoid. Finally, *d* shows an example which contains a wall that is too high for Mario to jump over, and therefore, has created a level with no valid solution. It could be argued that of the possible issues discussed thus far, this is the most severe as the previous two only impacted visual details, while this mistake undermines the very objective of the game itself.

One of the reasons we choose the WFC algorithm for this work is because it is capable of addressing the issues presented in examples *b* and *c* as both result from an inability to appropriately place tiles within the context of their neighbours: it is never the case that the top right-corner of a warp pipe should be placed under the top left-hand corner, or that a question block should ever be placed such that it’s bottom side is inaccessible to the player. If we are under the assumption that the sample input provided to the WFC algorithm does not contain either of these two errors, our resulting output is guaranteed to be free of them as well due to the constraint satisfaction being performed during its propagation step; in fact, the only way the algorithm can fail to generate a level is when the propagation step terminates due to a conflict arising from either of these two scenarios, in which case, the algorithm is typically reinitialized and reran.

As far as WFC can provide us with the confidence that the levels it produces will not contain tile configurations outside of those present in the original sample, there are no such guarantees that a level will always be necessarily playable as it is shown in example *d* which is a feasible output for WFC to produce. It may be tempting to suggest that a solution to this problem would be to increase the $N \times N$ pattern size used when sampling patterns, similar to how one might increase the size of a contextual window N when working with N -gram models [8] or N -dimensional Markov Chains [21], with the intention of restricting the algorithm to only form the exact structural elements sampled from the original source, however, there are multiple potential issues with this as a solution:

- (1) This work aims to present a solution that supports both versions of the WFC algorithm, while we made use of the pattern-based implementation of the WFC algorithm, there also exists an arguably more popular tile-based version of the algorithm which only considers each tile’s most immediate neighbours, and as such, makes no use of the $N \times N$ pattern window in question.

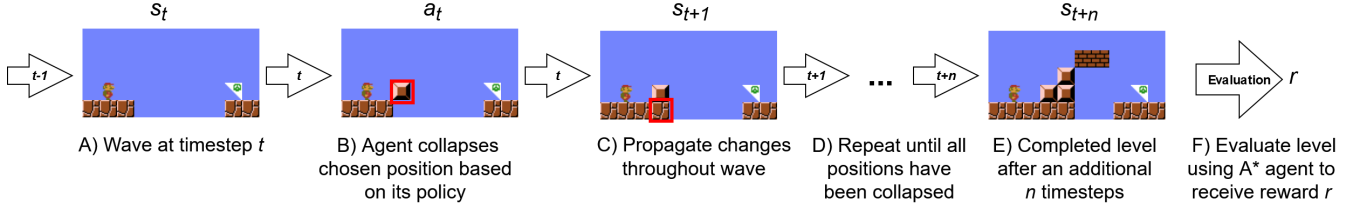


Figure 2: The process of generating a level using WFC framed as a RL task with timesteps, states, actions, and rewards indicated at each step A-F. Step A begins with an arbitrary wave with several positions already collapsed. Newly collapsed positions in steps B and C are indicated using a red outline.

- (2) Expanding pattern size reduces the expressive range of the generator. The most extreme example of this would be to increase the pattern size until it samples the entire source image, resulting in an output space containing only a single solution: the original source image.
- (3) The construction of *SMB* levels assume that Mario can only jump 4 tiles, and as such, one could increase the pattern size to 5x5 such that all obstacles placed are passable; and while this seemingly solves the issue for *SMB*—assuming that some cleverly crafted counterexamples don’t exist—it is not that case that this solution is guaranteed to work for all tile-based games. Entries to the *The Legend of Zelda* (Nintendo, 1986) series feature large overworlds, and meticulously crafted dungeons that would be impossible to guarantee playability for simply by increasing the size of our sampling pattern. Ideally, we would like to present a solution that works for as many games as possible, and as a result, will not be relying on any inherent structural conveniences provided by *SMB*, and will instead provide, what is hopefully, a more generalized environment-agnostic approach.

With these considerations in mind, let us revisit the aforementioned novel contribution of this work, which is the integration of WFC into the agent-environment interaction loop provided by RL. In a typical RL scenario, an agent’s behaviour is modulated by a reward signal returned by the environment, often positive when the agent performs an action that leads to a desirable state, and negative when it does not. By reframing the heuristic used during WFC’s *collapsing* step as an agent, and its resulting *propagation* step as a part of the agent’s environment, we now can easily define a reward function which reflects whatever desirable qualities we wish to see in the resulting levels produced by our agent. For the purposes of this work, this will simply be whether or not this method can be used to learn a heuristic which guides the WFC algorithm towards the production of levels that include valid solutions. There certainly are more complex reward functions that could be crafted which would address many issues we deem outside of the scope of this work, such as those which concern themselves with the production of levels that maximize player enjoyment, as there is undoubtedly a large gap that exists between levels which are merely playable and those which players will find fun and engaging.

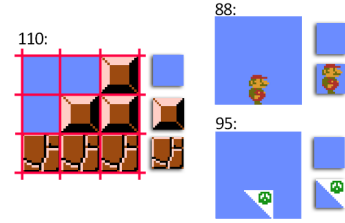


Figure 3: Examples of 3x3 tile patterns sampled from a modified version of World 1-1 from the original SMB with the individual tiles which compose them found to the right. A total 155 patterns were sampled. When generating a new level, the wave is initialized with pattern 88 (the player) in the bottom left-hand corner of the map, and pattern 95 (the goal) in the bottom right-hand corner.

3.2 Training Scenarios and Optimization

For our solution, we begin by setting up a new environment for our agent. The first step in this process is to sample a simplified version of World 1-1 from the original SMB. For this work, we will only be concerned with evaluating levels on the condition of whether or not they possess a valid solution path, and as such, our simplified version of World 1-1 only removes enemy tiles. In total, we sample 155 unique 3x3 tile patterns, with examples of these patterns shown in Figure 3. Next, we instantiate a new wave and partially collapse some of its positions: we fully collapse the bottom left-hand corner of the wave with pattern 88 which contains the Mario tile, and the bottom right-hand corner with pattern 95 which contains the goal flag tile, and finally, we partially collapse all other positions to remove any tiles that contain patterns that include either Mario or a goal flag tile as we only require one of each per level. This initial wave is considered to be the initial state s_0 . We then provide this initial state to our agent and receive our first action a_0 that specifies an uncollapsed position in the wave and proceed by collapsing based on our agent’s policy. Next, the environment updates the wave with the agent’s changes and propagates them to receive the next state s_{t+1} . This loop continues until the wave has been completely collapsed or a conflict during the propagation step occurs. Figure 2 illustrates this process beginning with a partially collapsed wave at an arbitrary timestep t and ends with the evaluation of a completed level to obtain reward r . This level evaluation is provided by the

Mario AI framework, and is performed by Robin Baumgarten’s A*-based agent (the winner of the Mario AI Competition [9]). If the agent successfully navigates the level the agent receives a reward of 1, but if the agent cannot complete the level, or a conflict is reached during propagation, the agent receives a reward of 0 for that level. Finally, in order to ensure that the agent is producing quality levels on a consistent basis, we have it generate a large number of levels and return the percentage of playable levels as the reward for that episode; this of course means that the maximum fitness score an agent can achieve is 1.0 meaning 100% of the levels it generated were indeed playable.

In terms of an optimization strategy for our agent, we chose an ES due to the fact that they can handle sparse and delayed rewards well, as we will only be using the total return for an episode. While methods like DQN have been successfully applied to sparse reward environments as well, we know that they can also be very sample inefficient as exemplified by the number of frames required to learn Atari 2600 games [7]; and as such, we decide to use an ES as it has been shown in [16] to be a competitive alternative to these approaches while providing some other advantages such as faster training times through the use of parallelization.

3.3 Generative Agents

For most PCG works, it is important that the generative method be capable of producing multiple outputs that vary in composition, yet remain consistent in quality. The original minimal entropy heuristic we are replacing would stochastically select tile patterns based on their likelihood of appearing in the original sample which allowed for a vast number of outputs given only a single starting state. Since we will be replacing this heuristic with deterministic models, we must introduce new mechanisms for producing outputs that vary in composition, and accomplish this using two distinct representations: the first approach has the agent choose an available pattern via a weighted random distribution it produces based on the current state of the wave s_t ; the second, randomizes the initial state s_0 , an approach taken in both PCGRL and in GANs which rely on a vector of Gaussian noise as input. We refer to these two approaches respectively as **Narrow**, and **Wide** to remain consistent with the complementary problem representations presented in the original PCGRL paper:

- **Narrow:** In this representation, the agent is given the lowest entropy wave position to collapse, and based on the current state of the wave, constructs a weight vector w which will be randomly sampled to select a pattern. The action space for this agent is discrete with values ranging from 0 to $n - 1$, where n is the number of patterns sampled from the source level (155 in our experiments).
- **Wide:** This representation’s wave is initialized with r randomly collapsed positions in order to promote variations in output as each action does not randomly sample a weighted distribution like its **Narrow** counterpart does, rather, it is given full control over both the position that should be collapsed as well as which pattern will be assigned to it. The agent itself outputs a 3D weight matrix that shares the desired width and height of the output level but has a depth equal to the number of patterns sampled from the source

image. The agent’s policy will collapse the wave position with the largest sum along the depth axis, and places the pattern with the largest value along it. The action space for this agent is also discrete with values x and y denoting the position of the wave that is to be collapsed and another ranging from 0 to $n - 1$, where n is the number of patterns sampled from the source level (155 in our experiments).

The input to both of our agents is a binary representation of the current wave, where 1 represents a pattern that is still possible at a given position, 0 representing that the pattern is not. We also apply this as a mask to the output of the **Wide** agent such that it cannot select patterns that have already been ruled out by the propagation step, or positions that have already been collapsed in previous iterations.

3.4 Agent Network Architectures and Training

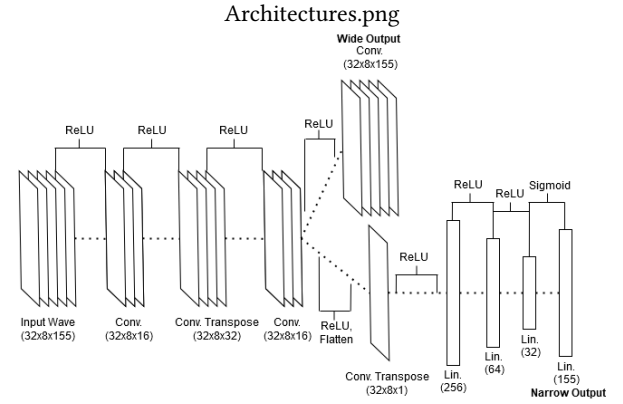


Figure 4: The network architectures for both the **Narrow** (bottom output) and **Wide** (top output) agents.

Both of the agents we present in this work receive the current wave at time t as input in the form of a binary matrix. The dimensions of these waves are based on the size of the levels we will be generating and the number of patterns sampled from the source level. In our experiments, we decided to build levels with a width of 32 and height of 8. It should be noted that these dimensions can be adjusted to any output size desired, however, larger levels will result in longer training times. Finally, the number of patterns sampled from our simplified World 1-1 is 155, resulting in a wave of size 32x8x155.

Figure 4 presents the network architecture for both the **Narrow** and **Wide** agents. Because these models are accepting 3-Dimensional waves as input, we use a series of convolutional layers to process them. Where our two models differ is in their output; the **Wide** model simply runs an additional convolutional layer with a depth of 155, while the **Narrow** model flattens its output into a series of linear layers that ultimately end in an output of size 155.

To train these agents we use the ES presented in [16]. The algorithm is described as a black-box optimization method that optimizes a function $f(w)$ with respect to w , the parameters of our DCNNs. It is referred to as a black-box method because it does not

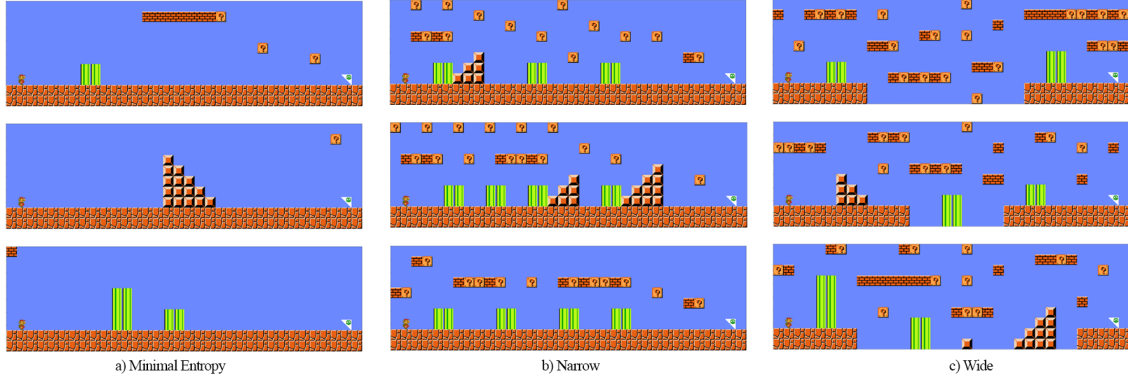


Figure 5: Three example levels generated using a) the minimal entropy heuristic, b) our Narrow agent, and c) our Wide agent.

need to know the specifics of $f(w)$ in order to optimize it, only that it accepts the parameters of the network as input and returns a reward r . By finding the parameters w which optimize this function, we have effectively trained a model to perform whatever task is being specified by $f(w)$. In this work, $f(w)$ is one episode of training where we generate 60 levels using the WFC algorithm with one of our agents acting as a heuristic. The reward r is the percentage of levels that were successfully completed by the A* agent provided in the Mario AI framework.

In terms of hyperparameters we used for training, we train our agents for 10 generations using a population size of 7, a noise standard deviation sigma of 0.05, and a learning rate alpha of 0.01.

4 RESULTS

	Entropy	Narrow	Wide
Avg. Num. Jumps	1.6	3.3	3.0
Avg. Max Jump X	86.6	113.1	107.4
Avg. % Completed	91.0%	100.0%	99.0%
% Playable	84.8%	100.0%	97.2%

Table 1: Performance metrics obtained from 250 levels generated by the WFC algorithm using minimal entropy, Narrow and Wide heuristics.

To measure how well our heuristics perform, we generate 250 levels using the WFC algorithm with the original minimal entropy heuristic, and our **Narrow** and **Wide** representations. We will be gathering a number of performance metrics using the same A* agent used during training, including: the average number of jumps needed to complete a level, the average maximum horizontal jump distance, the average percentage of the level that was completed, and the percentage of levels completed by the agent.

Results gathered from all three heuristics are summarized in Table 1. Most applicable to this work are the last two rows which inform us on the playability of the levels generated by a specific method. We include the number of jumps performed by the agent as well as the agent’s maximum horizontal jumping distances as these are popular metrics used in assessing the quality of SMB

levels; as an example, [2] uses the number of jumps performed as a measure of a level’s linearity, while [25] which uses it as a measure of difficulty. This work will not operate under the assumption that higher values for both of these jump-related metrics necessarily correlate with an increase in a level’s overall quality, but we simply provide them as supplementary metrics. This said, based on the results presented in Table 1, both our **Narrow** and **Wide** agents achieved higher scores than the benchmark values set by the original minimal entropy heuristic in all four performance metrics. In the following subsections, we will discuss each of these results individually along with some examples of the levels generated by each heuristic in order to gain some insight as to why this is the case.

4.1 Minimal Entropy

Examples of the levels generated in our experiments are presented in Figure 5. We can see the levels generated using minimal entropy are relatively sparse compared to those generated using our **Narrow** and **Wide** agents. This explains why the AI agent performed a lower number of jumps and horizontal jumping distance as there are far fewer obstacles in the levels generated using this heuristic. In terms of playability, this heuristic generates solvable levels approximately 85% of the time. This is likely due to the height of obstacles such as warp pipes being too tall (such as the bottom example), or the direction of staircases being constructed with the wrong orientation (such as the middle example). Misplaced obstacles such as these form impassable barriers for the player, and result in a reduction to this heuristic’s playability score.

4.2 Learned Heuristics

Levels generated using the **Narrow** and **Wide** agent showed an increase in all of the four performance metrics, most notably in the likelihood of generating playable levels. When examining Figure 5, the levels generated for these two heuristics have considerably more obstacles than those generated using minimal entropy. It could be argued that the levels generated using our heuristics contain too many objects, and as a result, are less aesthetically pleasing than the ones generated using minimal entropy. This overcrowding is undoubtedly a result of our simple reward function, which is only concerned with whether or not the level is playable. Future works

should experiment with different reward functions in an attempt to address these issues while maintaining valid solutions to the levels generated. One such reward function may wish to include the occurrence count of each pattern in the resulting output and penalize the agent for not matching the probability of each pattern being found in the original source, thereby incentivizing the agent to place patterns with the same frequency as the original source level. This said, we are pleased that both our agent-based heuristics succeeded in achieving their objective of generating more playable levels. We are convinced this is due to a behavioural tendency of limiting the height of objects such as Warp Pipes and stairs exhibited by both agents; an exception being in the bottom image of the **Wide** agent, where the first Warp Pipe is too tall to jump over. This example helps to provide some insight to an important question: why are all of the levels generated by the **Narrow** agent playable, yet only 97% of those generated by the **Wide** agent found to contain valid solutions? We can see from this example that the **Wide** agent is still generating some obstacles that are too tall and we suspect that this is due to the vast difference in output dimensions of these two agents' networks. Where the **Wide** agent had an output of size 39,680 (output width x output height x number of patterns) to provide enough information to its policy to both select a wave position as well as select an appropriate pattern, the **Narrow** agent was tasked with only the latter, resulting in a much smaller output of 155 (the number of possible patterns). We surmise that the task being performed by the **Wide** agent was much more challenging than that of the **Narrow** agent. It is possible that the **Wide** agent requires more than 10 generations of training to raise the likelihood of generating playable levels to our target of 100%.

5 CONCLUSION

5.1 Summary

This work has shown a novel approach to learning agent-based heuristics for the WaveFunctionCollapse algorithm for the purpose of generating playable video game environments. We have shown two different representations of the problem inspired by those from the original PCGRL paper: a **Narrow** approach, where the agent learned to construct a vector of weights to be sampled from randomly when selecting patterns; and a **Wide** approach, where the agent both selects a wave position to collapse as well as an appropriate pattern. These models were trained to generate levels for the original *Super Mario Bros.*, and were evaluated using a game-playing agent provided by the *Mario AI Competition Framework*. The reward signal used for training our agents was a simple percentage of the number of levels successfully navigated by this A* agent. The optimization of our two agents was performed by an ES and we ultimately find an increase in the percentage of playable levels generated using our learned heuristics over those using WFC's original minimal entropy heuristic. As the WFC algorithm continues to gain popularity amongst both PCG researchers and commercial game-developers, we hope that this work can contribute in widening the practicality of the algorithm by demonstrating how it can reliably generate playable levels for many genres of games.

5.2 Future Work

While this work presented how heuristics for the WFC algorithm can be obtained using RL, there are still a number of future works that can be conducted. The first of which is to train these **Narrow** and **Wide** heuristics using algorithms such as Q-Learning, DQN, or PPO as it is possible that these algorithms may prove to be more effective than the ES used in this work. The adaptation of the approach taken in this work to accommodate these methods should be minimal as we did strictly adhere to the standard RL-framework.

There is also an opportunity to experiment with different reward functions. We chose to use one of the simplest reward functions possible which rewarded the agent based on the number of playable levels it was able to generate. Our motivation was to experiment with a reward function that could be applicable to almost any genre of game, however, more game-specific reward functions could be crafted to obtain better results, or to generate levels that contain specific structural features or prompt certain actions from the player. Using SMB as an example, should we desire to generate levels with more obstacles, a reward function could be crafted that accumulates the number of jumps performed by the player. Other reward functions may also be designed to accommodate more complex tilesets or genres of games. As this work was only concerned with generating levels that contain valid solutions, we reduced our tile set to only include static obstacles by removing enemies; in the future, we would like to see experiments conducted using expanded tilesets which include dynamic elements such as enemies and craft new reward functions which force interactions between them and the player. Finally, experimenting with games from other genres, may require interesting reward functions unique to their genre that go beyond the simple binary reward presented in this work.

REFERENCES

- [1] Alejandro Baldominos, Yago Saez, Gustavo Recio, and Javier Calle. 2015. Learning Levels of Mario AI Using Genetic Algorithms. In *Proceedings of the 16th Conference of the Spanish Association for Artificial Intelligence on Advances in Artificial Intelligence - Volume 9422*. 267–277.
- [2] Steve Dahlskog, Julian Togelius, and Mark J. Nelson. 2014. Linear levels through n-grams. In *Proceedings of the 18th International Academic MindTrek Conference on Media Business, Management, Content & Services*. 200–206.
- [3] Clara Fernández-Vara and Alec Thomson. 2012. Procedural Generation of Narrative Puzzles in Adventure Games: The Puzzle-Dice System. In *Proceedings of the The third workshop on Procedural Content Generation in Games*. 12.
- [4] Michael Cerny Green, Ahmed Khalifa, Gabriella A. B. Barros, Andy Nealen, and Julian Togelius. 2018. Generating levels that teach mechanics. In *Proceedings of the 13th International Conference on the Foundations of Digital Games*. 55.
- [5] Jason Grinblat and C. Brian Bucklew. 2017. Subverting historical cause & effect: generation of mythic biographies in Caves of Qud. In *Proceedings of the 12th International Conference on the Foundations of Digital Games*. 76.
- [6] Maxim Gumin. 2016. Bitmap & tilemap generation from a single example by collapsing a wave function. <https://github.com/mxgmn/WaveFunctionCollapse>
- [7] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Daniel Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. 2017. Rainbow: Combining Improvements in Deep Reinforcement Learning. In *AAAI*. 3215–3222.
- [8] Daniel Jurafsky and James H. Martin. 2006. *Speech and Language Processing*.
- [9] S. Karakovskiy and J. Togelius. 2012. The Mario AI Benchmark and Competitions. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 55–67.
- [10] Isaac Karth and Adam M. Smith. 2017. WaveFunctionCollapse is constraint solving in the wild. In *Proceedings of the 12th International Conference on the Foundations of Digital Games*. 68.
- [11] Barbara De Kegel and Mads Haahr. 2019. Procedural Puzzle Generation: A Survey. *IEEE Transactions on Games* (2019), 1–1.
- [12] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. 2020. PCGRL: Procedural Content Generation via Reinforcement Learning. *Proceedings of the*

- AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment 16, 1 (2020), 95–101.
- [13] Ricardo Lopes, Elmar Eisemann, and Rafael Bidarra. 2018. Authoring Adaptive Game World Generation. *IEEE Transactions on Computational Intelligence and AI in Games* 10, 1 (2018), 42–55.
 - [14] Pedro Minini and Joaquim Assuncao. 2020. Combining Constructive Procedural Dungeon Generation Methods with WaveFunctionCollapse in Top-Down 2D Games. In *Proceedings of SBGames*.
 - [15] Diego Perez-Liebana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius, and Simon M. Lucas. 2018. General Video Game AI: a Multi-Track Framework for Evaluating Agents, Games and Content Generation Algorithms. *arXiv preprint arXiv:1802.10363* (2018).
 - [16] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. *arXiv preprint arXiv:1703.03864* (2017).
 - [17] Hugo Scurti and Clark Verbrugge. 2018. Generating Paths with WFC.. In *AIIDE*. 271–274.
 - [18] Noor Shaker, Miguel Nicolau, Georgios N. Yannakakis, Julian Togelius, and Michael O'Neill. 2012. Evolving levels for Super Mario Bros using grammatical evolution. In *2012 IEEE Conference on Computational Intelligence and Games (CIG)*. 304–311.
 - [19] Noor Shaker, Julian Togelius, and Mark J. Nelson. 2016. *Procedural Content Generation in Games*.
 - [20] A. M. Smith and M. Mateas. 2011. Answer Set Programming for Procedural Content Generation: A Design Space Approach. *IEEE Transactions on Computational Intelligence and AI in Games* 3, 3 (2011), 187–200.
 - [21] Sam Snodgrass and Santiago Ontañón. 2014. Experiments in map generation using Markov chains.. In *FDG*.
 - [22] O. Stalberg. 2016. Wave Function Collapse in Bad North. <https://youtu.be/0bcZb-SsrA>
 - [23] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgard, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. 2018. Procedural Content Generation via Machine Learning (PCGML). *IEEE Transactions on Games* 10, 3 (2018), 257–270.
 - [24] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne. 2011. Search-Based Procedural Content Generation: A Taxonomy and Survey. *IEEE Transactions on Computational Intelligence and AI in Games* 3, 3 (2011), 172–186.
 - [25] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M. Lucas, Adam Smith, and Sebastian Risi. 2018. Evolving mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the Genetic and Evolutionary Computation Conference on*. 221–228.